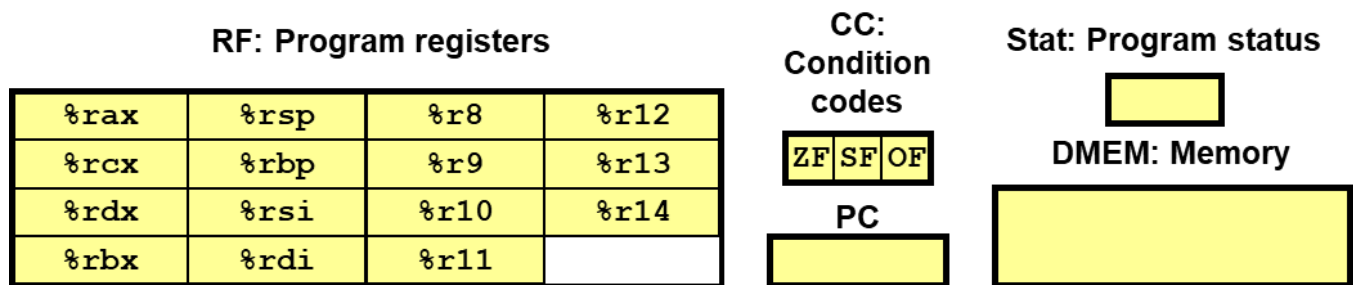


Lecture 4 Processor Architecture

Processor Architecture

We introduce the processor architecture Y86-64, simplified from X86-64.

Y86-64 Processor State



- Program registers
 - 15 registers(omit %r16 , only for x86), each 64 bit
- Condition codes
 - single-bit flags by arithmetic or logical instructions.
- Program Counter
 - Indicate address of **next instruction**.
- Program status
 - Indicates either **normal operation** or some **error condition**
- Memory
 - **Byte-addressable** storage array.

Y86-64 Instruction Set

Byte	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
cmovXX rA, rB	2	fn	rA	rB						
irmovq V, rB	3	0	F	rB	V					
rmmovq rA, D(rB)	4	0	rA	rB	D					
mrmmovq D(rB), rA	5	0	rA	rB	D					
OPq rA, rB	6	fn	rA	rB						
jXX Dest	7	fn	Dest							
call Dest	8	0	Dest							
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						

- Format
 - 1-10 bytes of information read from memory.
 - Can determine instruction length from first byte
 - Simpler encoding than x86-64
 - Each accesses and modifies some part of the program.

Encoding Register

Each register has 4-bit ID

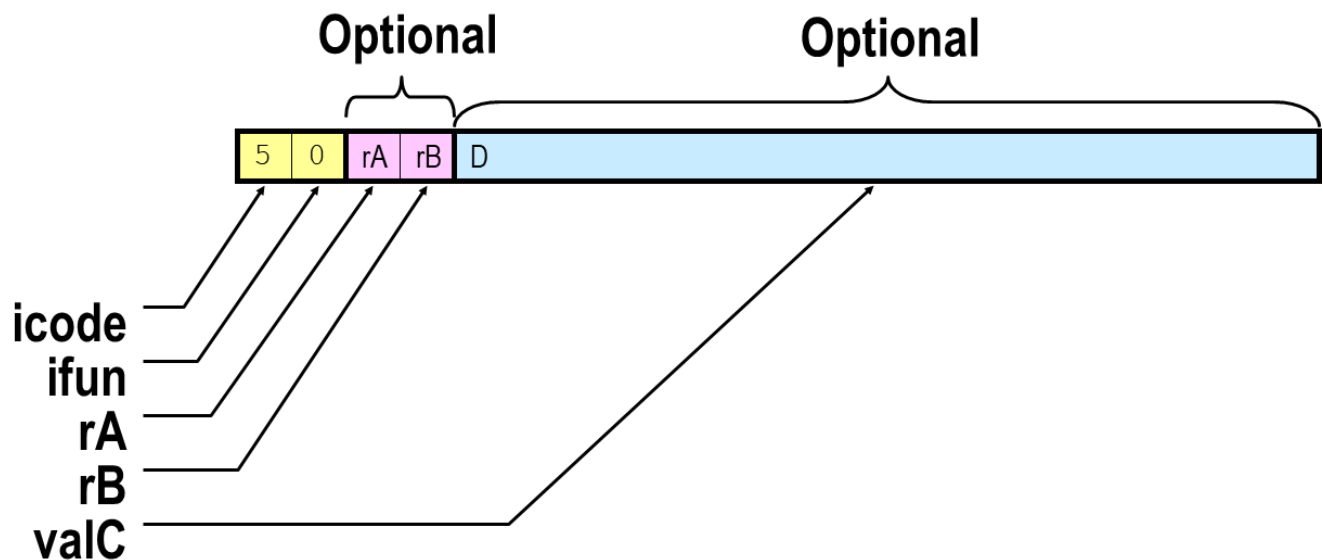
<code>%rax</code>	0
<code>%rcx</code>	1
<code>%rdx</code>	2
<code>%rbx</code>	3
<code>%rsp</code>	4
<code>%rbp</code>	5
<code>%rsi</code>	6
<code>%rdi</code>	7

<code>%r8</code>	8
<code>%r9</code>	9
<code>%r10</code>	A
<code>%r11</code>	B
<code>%r12</code>	C
<code>%r13</code>	D
<code>%r14</code>	E
No Register	F

Y86-64 Little-endian Encoding

As with x86-64, all integers have **little-endian encoding**.

Instruction Decoding



Instruction format

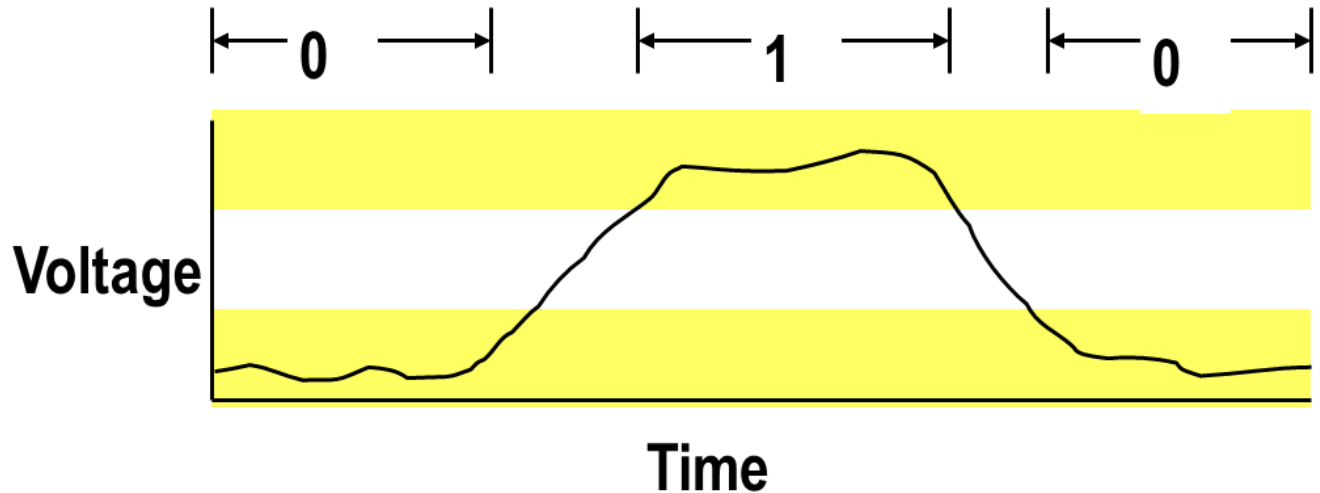
- Instruction byte: `icode:ifun`
- Optional register byte: `rA:rB`
- Optional constant word: `valC` (immediate or address, so on)

Classwork

problem 4.1

Digital Signals

Use voltage thresholds to extract discrete value from continuous signal.



Computing with Logic Gates

- Output are Boolean functions of inputs
- Respond continuously to changes in inputs with some small delay

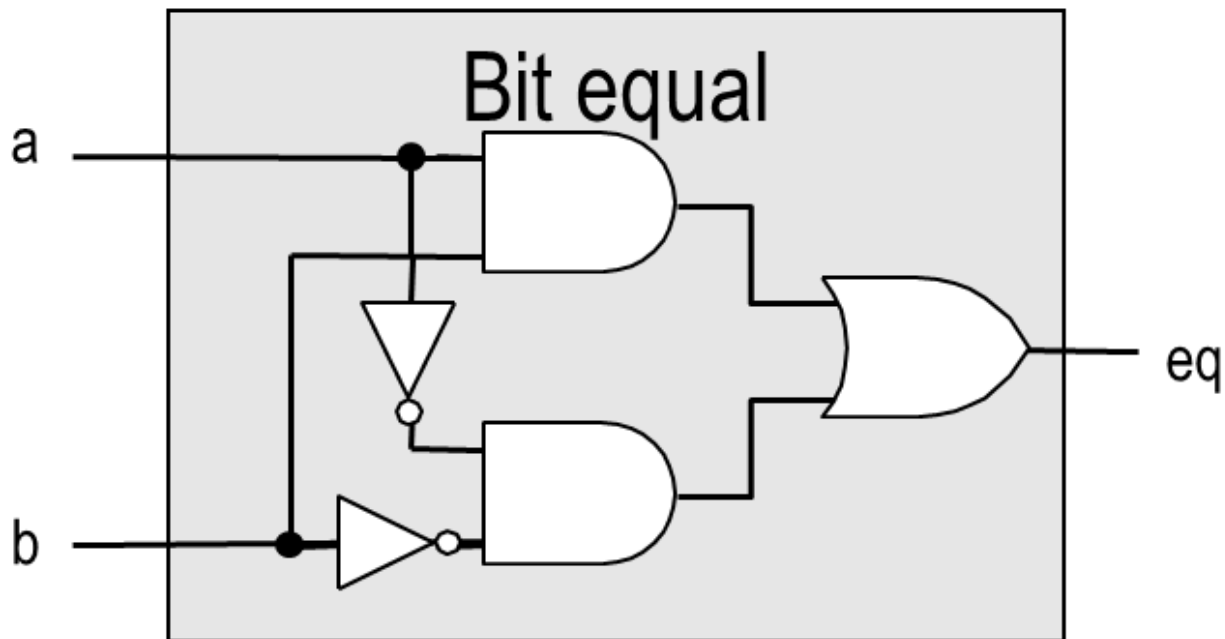
Like, AND gates, OR gates, NOT gates.

Combinational Circuits

By assembling a number of logic gates into networks, we can construct computational blocks known as combinational circuits with following characteristics:

- Every logic gate input must be connected to exactly one of the following:
 - one of the system inputs(primary input)
 - the output connection of some memory element
 - the output of some logic gate
- The outputs of two or more logic gates cannot be connected together. (Two different voltages possibly causes an invalid voltage or circuit malfunction)
- The network must be acyclic. That is, continuously responds to changes on primary inputs and primary outputs become Boolean functions of primary inputs.

Bit Equality



HCL Expression:

```
bool eq = (a && b) || (!a && !b)
```

different from C, the `=` represents to rename right expression.

HCL(hardware control language) is very simple hardware description language. And Boolean operations have syntax similar to C logical operations

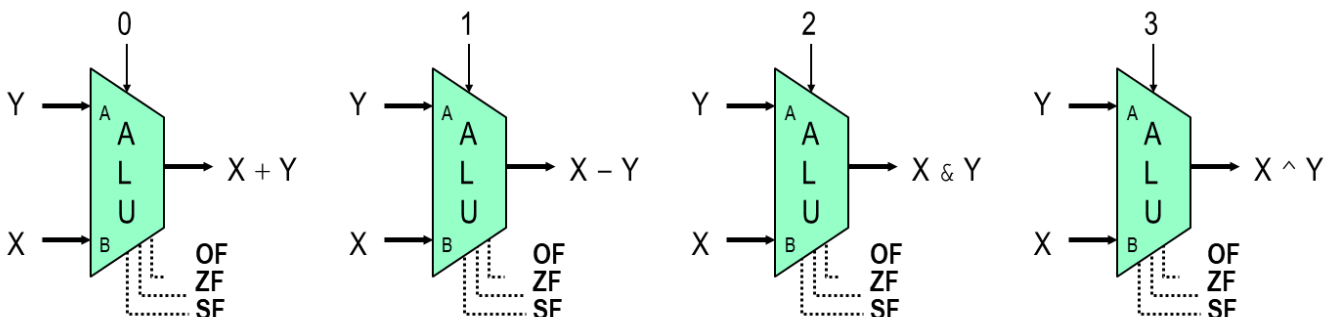
Word Equality

Use more **bit equality** to check for every pair of bit.

```
bool Eq = (A == B)
```

Arithmetic Logic Unit

- **combination logic**: continuously responding to inputs
- control signal selects function computed(4 arithmetic/logical operations in Y86-64)
- also computes values for condition codes

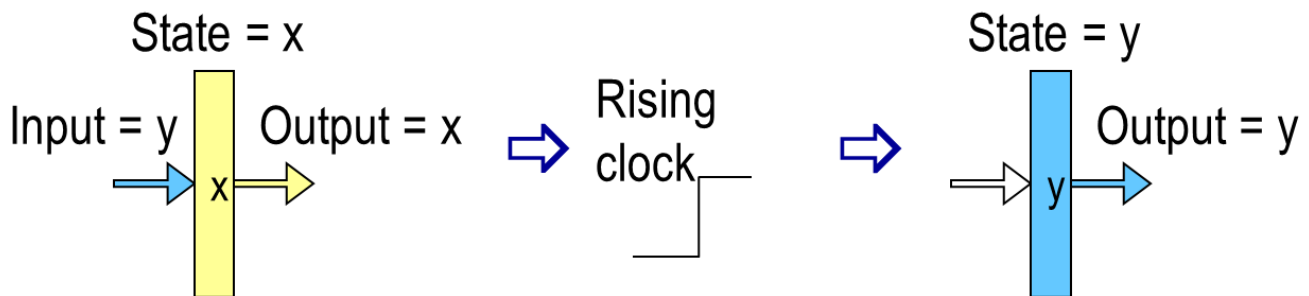


Sequential Circuits

Systems that can store information, or, have state and perform computations on that state. It is controlled by a single clock, a periodic signal that determines when new values are to be loaded into devices.

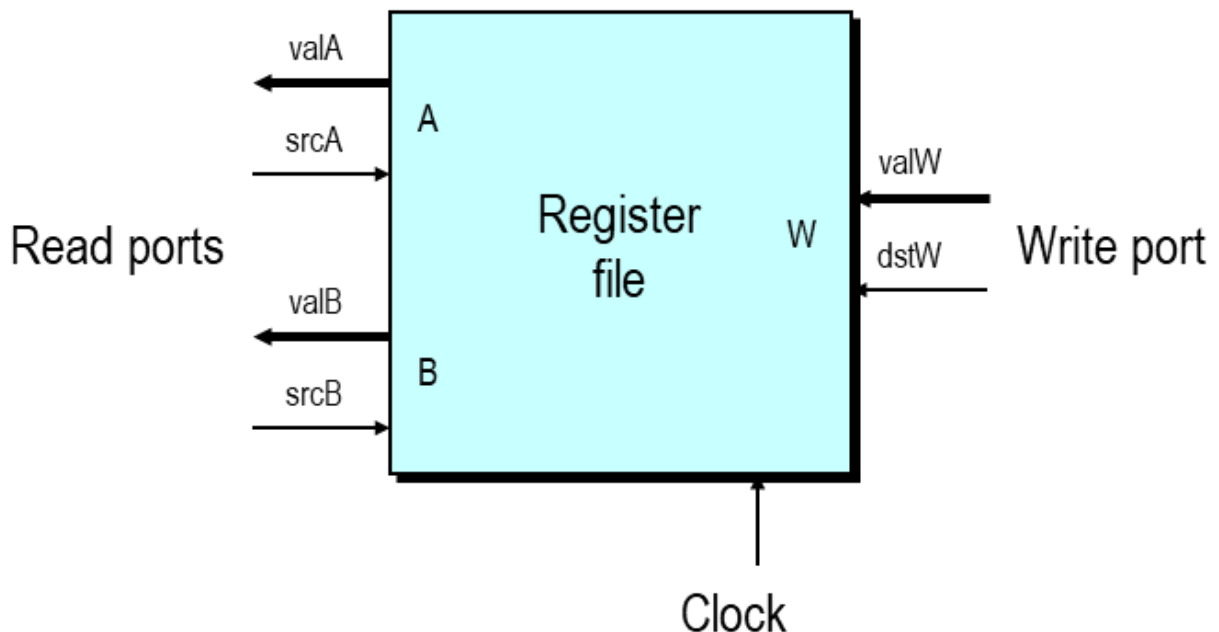
Register Operation

The register stores data bits. For most of time acts barrier between input and output. As clock rises, loads input.



Register File

Register file is a set of registers, stores multiple words of memory.



In register file, register identifier serves as address.

For read ports, srcA and srcB pass the addresses of corresponding registers and output the data valA and valB in corresponding register. It is like combinational logic, output data generated based on input address after some delay.

For write ports, input the address `dstW` and input the data to be written `valW`. It is like `register`, update only as clock rises.

For `multiple ports`:

- can `read and/or write multiple words in one cycle`.
- can `read and write simultaneously`.

SEQ Stages

- Fetch
Read instructions from instruction memory(address storing in PC)
- Decode
Read program registers
- Execute
compute value or address
- Memory
Read or write data
- Write back
write program registers
- PC
Update program counter

PC

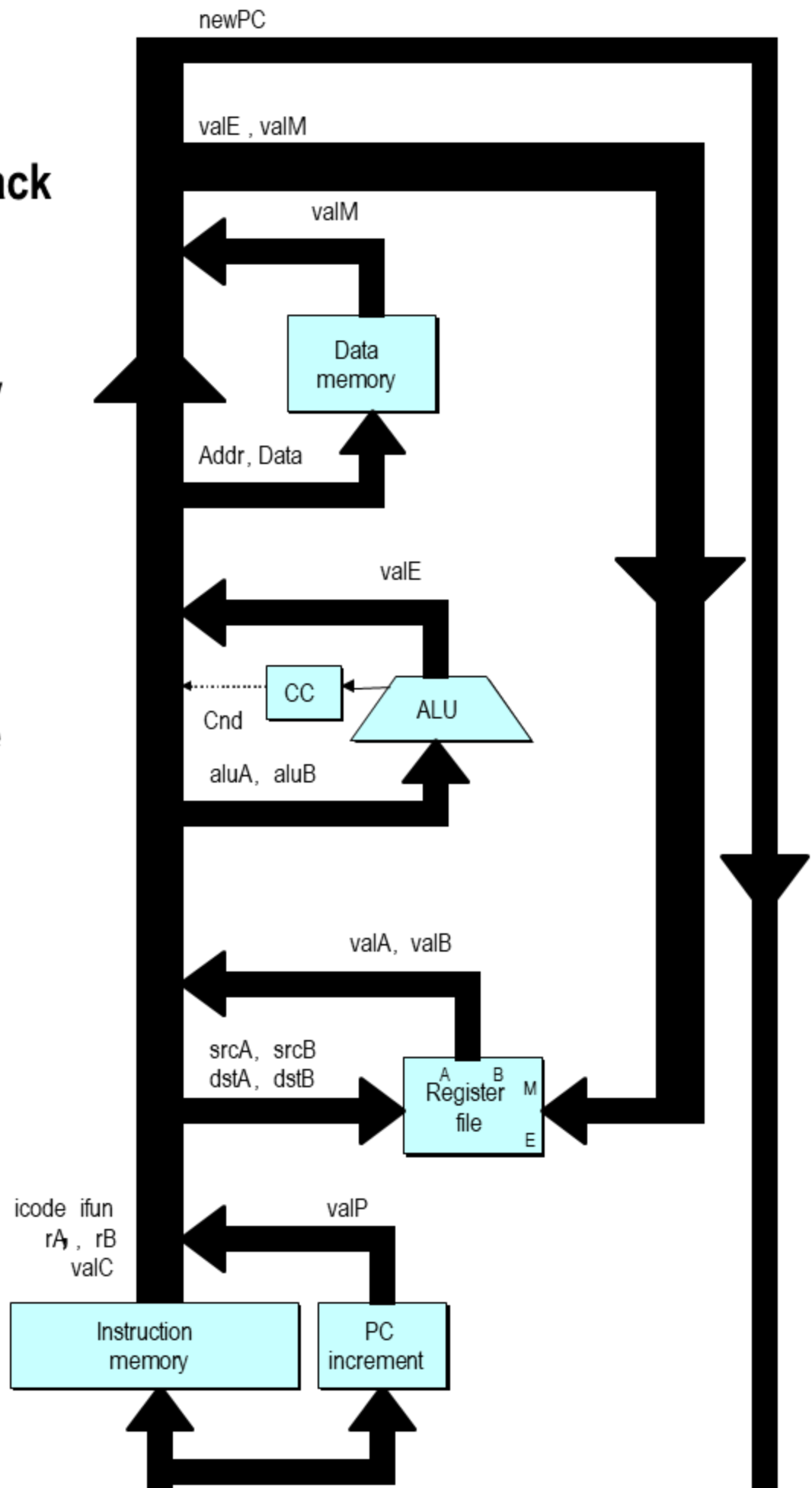
Write back

Memory

Execute

Decode

Fetch





Executing Arithmetic/Logical Operation

OPq rA, rB $\rightarrow$ 6 fn rA rB

- Fetch
 - Read 2 bytes
- Decode
 - Read operand registers
- Execute
 - Perform operation
 - set condition codes
- Memory
 - Do nothing
- Write back
 - update register
- PC update
 - increment PC by 2.

	OPq rA, rB	
Fetch	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$	Read instruction byte
	$\text{rA:rB} \leftarrow M_1[\text{PC}+1]$	Read register byte
	$\text{valP} \leftarrow \text{PC}+2$	Compute next PC
Decode	$\text{valA} \leftarrow R[\text{rA}]$	Read operand A
	$\text{valB} \leftarrow R[\text{rB}]$	Read operand B
Execute	$\text{valE} \leftarrow \text{valB OP valA}$	Perform ALU operation
	Set CC	Set condition code register
Memory		
Write back	$R[\text{rB}] \leftarrow \text{valE}$	Write back result
PC update	$\text{PC} \leftarrow \text{valP}$	Update PC

Example:

Computations for the Specific Instruction

For the subtract instruction in the following code:

```
0x000: 30f20900000000000000 | irmovq $9, %rdx
0x00a: 30f31500000000000000 | irmovq $21, %rbx
0x014: 6123 | subq %rdx, %rbx # subtract
```

Operation stages:

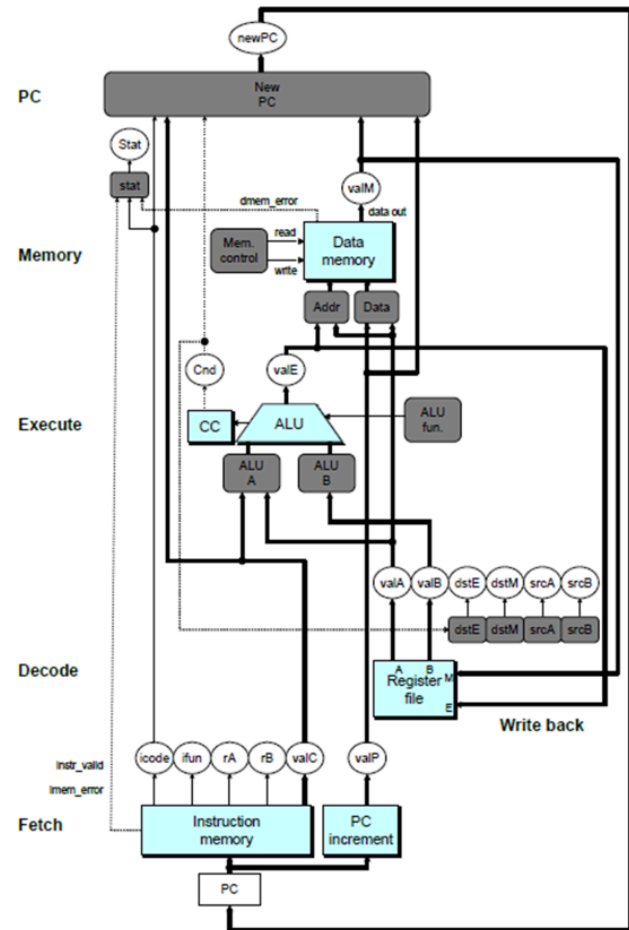
Stage	OPq rA, rB	subq %rdx, %rbx
Fetch	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{rA:rB} \leftarrow M_1[\text{PC} + 1]$ $\text{valP} \leftarrow \text{PC} + 2$	$\text{icode:ifun} \leftarrow M_1[0x014] = 6:1$ $\text{rA:rB} \leftarrow M_1[0x015] = 2:3$ $\text{valP} \leftarrow 0x014 + 2 = 0x016$
Decode	$\text{valA} \leftarrow R[\text{rA}]$ $\text{valB} \leftarrow R[\text{rB}]$	$\text{valA} \leftarrow R[\%rdx] = 9$ $\text{valB} \leftarrow R[\%rbx] = 21$
Execute	$\text{valE} \leftarrow \text{valB} \text{ OP } \text{valA}$ Set CC	$\text{valE} \leftarrow 21 - 9 = 12$ $\text{ZF} \leftarrow 0, \text{SF} \leftarrow 0, \text{OF} \leftarrow 0$
Memory		
Write back	$R[\text{rB}] \leftarrow \text{valE}$	$R[\%rbx] \leftarrow \text{valE} = 12$
PC update	$\text{PC} \leftarrow \text{valP}$	$\text{PC} \leftarrow \text{valP} = 0x016$

SEQ Hardware

SEQ Hardware

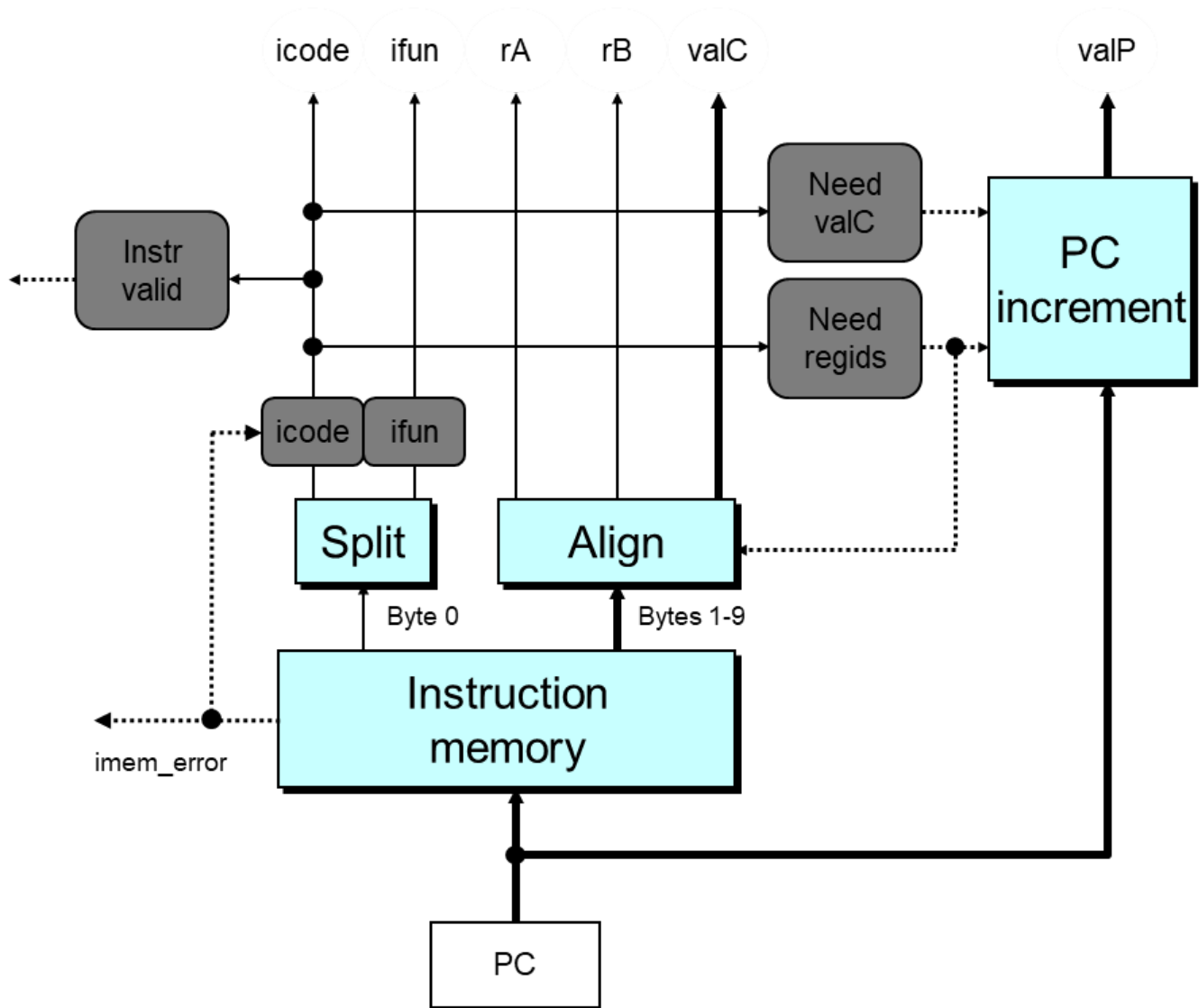
■ Key

- Blue boxes: predesigned hardware blocks
 - E.g., memories, ALU
- Gray boxes: control logic
 - Describe in HCL
- White ovals: labels for signals
- Thick lines: 64-bit word values
- Thin lines: 4-8 bit values
- Dotted lines: 1-bit values



Here is the total introduction of SEQ hardware and the following are different stages

Fetch and Write Back Logic



Predefined blocks:

- PC: Register containing PC
- Instruction memory: Read 10 bytes(signal invalid address)
- Split: Divide instruction byte into `icode` and `ifun`.
- Align: Get fields for `rA`, `rB` and `valC`.

For some instructions without operands `rA`, `rB` or `valC`, corresponding bytes are just aligned without being computed.

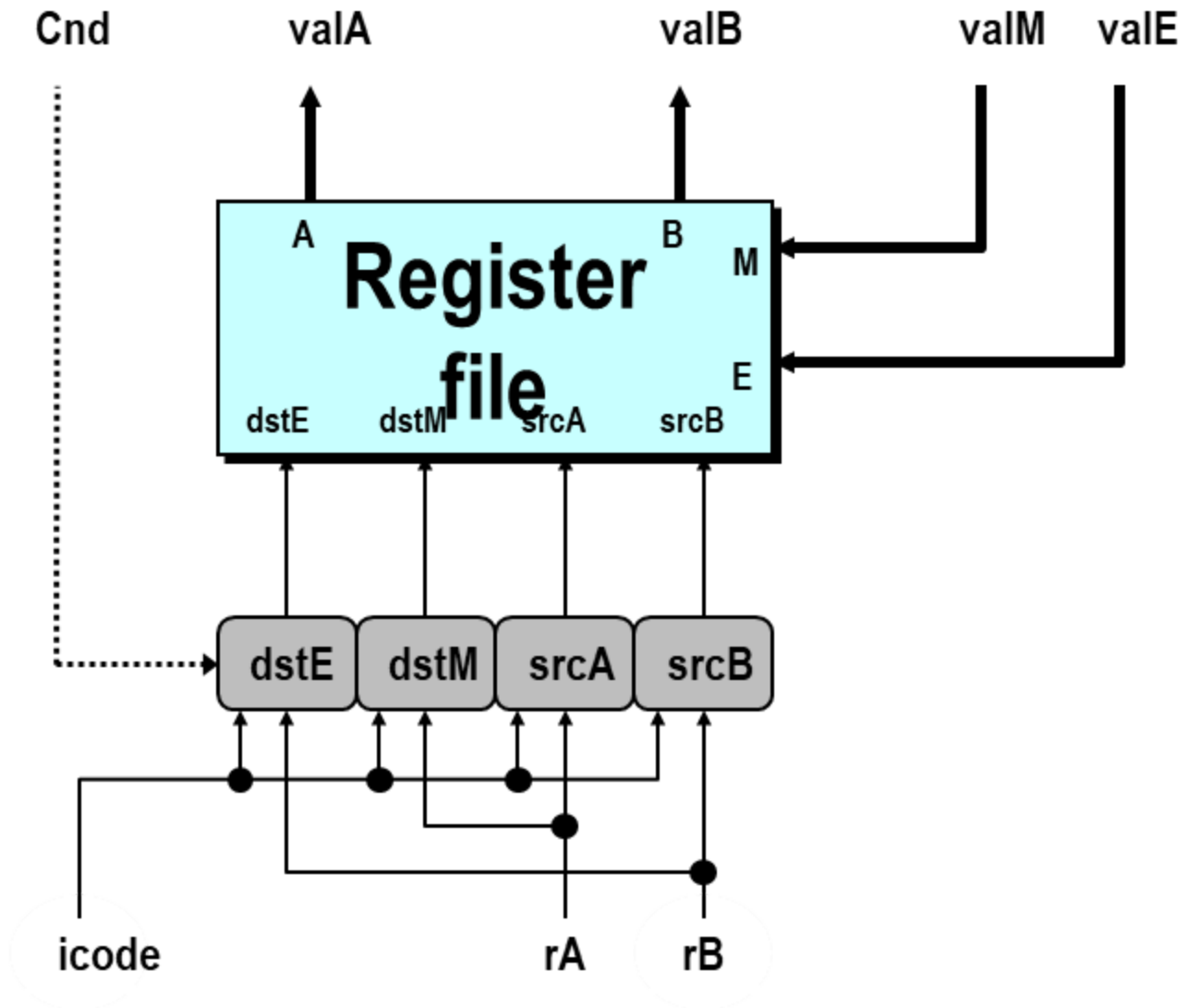
Control logic:

- Instr. Valid: Judge whether the instruction valid.
- Need regids: Judge whether this instruction have a register byte.
- Need valC: Judge whether this instruction have a constant word `valC`.

Suppose the output of Need regids and Need valC are respectively r and i , then

$$\text{valP} = \text{PC} + 1 + r + 8i.$$

Decode Logic



Register file

- Read ports A, B
- Write ports E, M
- Addresses are register IDs or 15.

Control logic:

- srcA , srcB : read port addresses
- dstE , dstM : write port addresses

Signals:

- Cnd: indicate whether or not to perform **conditional move**(computed in execute stage)

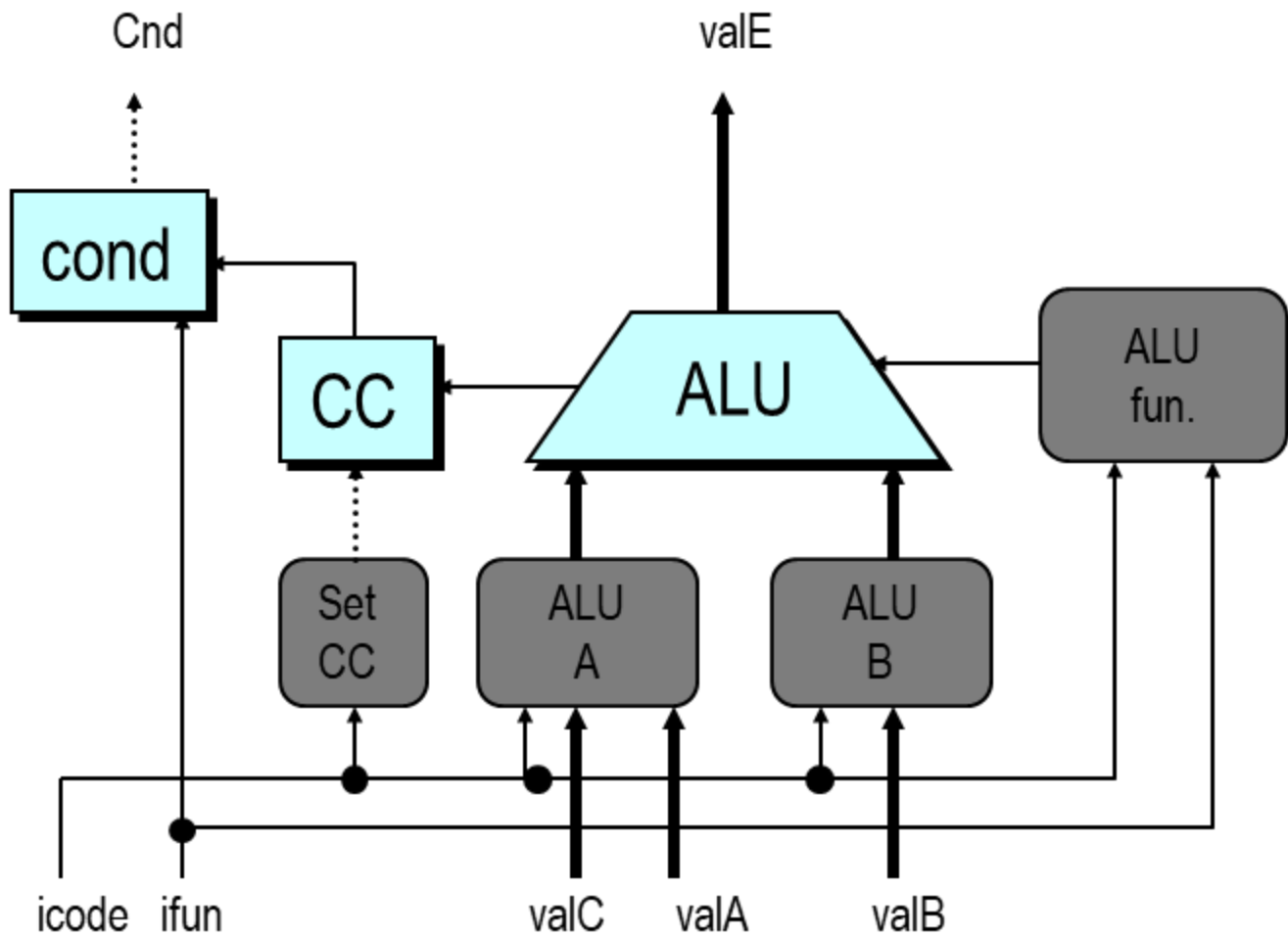
Read operations:

	OPq rA, rB	
Decode	valA $\leftarrow$ R[rA]	Read operand A
	cmovXX rA, rB	
Decode	valA $\leftarrow$ R[rA]	Read operand A
	rmmovq rA, D(rB)	
Decode	valA $\leftarrow$ R[rA]	Read operand A
	popq rA	
Decode	valA $\leftarrow$ R[%rsp]	Read stack pointer
	jXX Dest	
Decode		No operand
	call Dest	
Decode		No operand
	ret	
Decode	valA $\leftarrow$ R[%rsp]	Read stack pointer

write operations:

	OPq rA, rB	
Write-back	R[rB] ← valE	Write back result
	cmovXX rA, rB	
Write-back	R[rB] ← valE	Conditionally write back result
	rmmovq rA, D(rB)	
Write-back		None
	popq rA	
Write-back	R[%rsp] ← valE	Update stack pointer
	jXX Dest	
Write-back		None
	call Dest	
Write-back	R[%rsp] ← valE	Update stack pointer
	ret	
Write-back	R[%rsp] ← valE	Update stack pointer

Execute Logic



Unit:

- ALU
 - implements 4 required functions
 - generates condition code values
- CC
 - register with 3 condition code bits
- cond
 - compute conditional jump/move flag

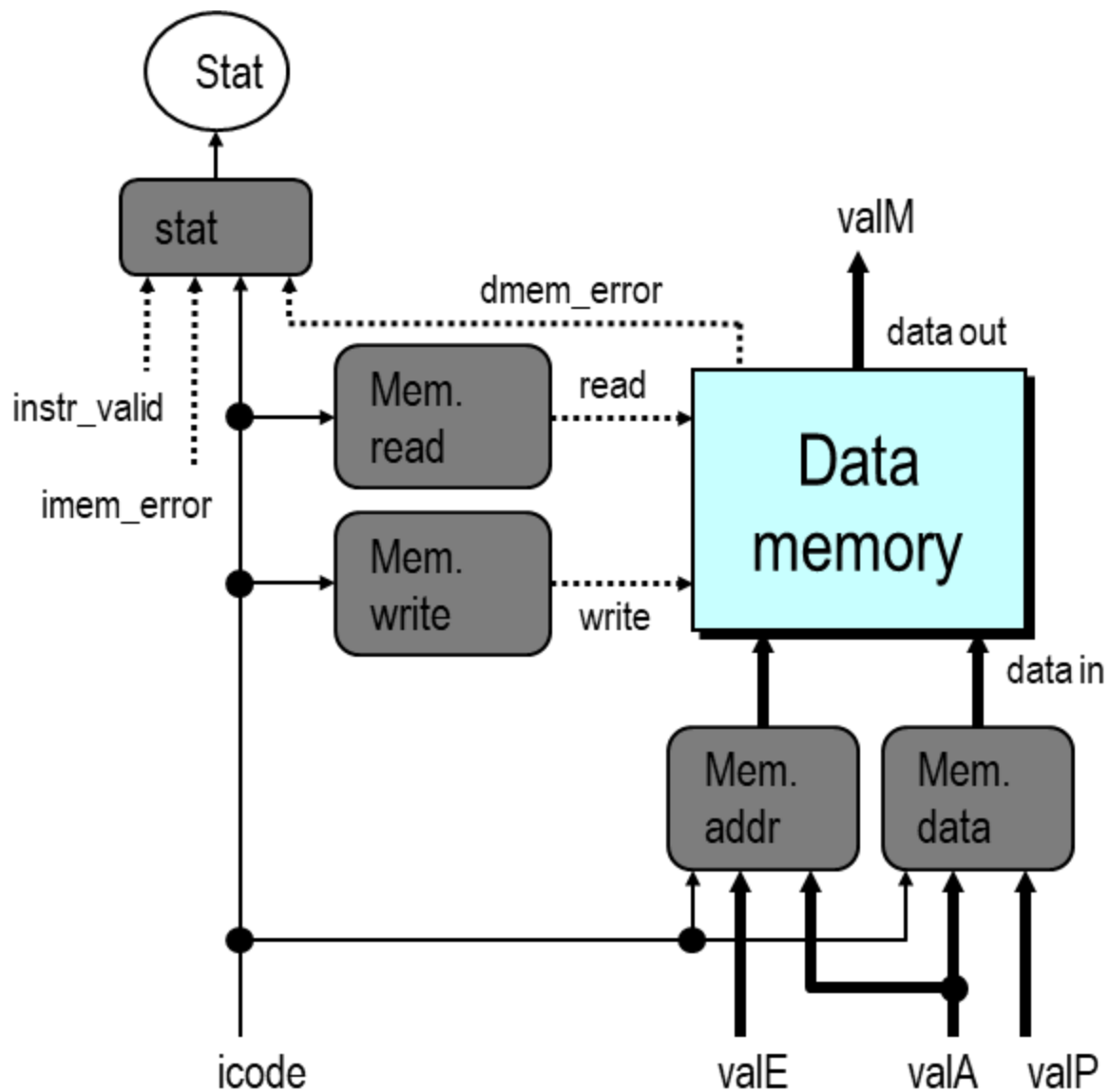
Control logic:

- Set CC: whether the condition code register should be loaded
- ALU A: input A to ALU
- ALU B: input B to ALU
- ALU fun: What function should ALU compute

Different ALU input and operations:

	OPq rA, rB	
Execute	$\text{valE} \leftarrow \text{valB} \text{ OP } \text{valA}$	Perform ALU operation
	cmovXX rA, rB	
Execute	$\text{valE} \leftarrow 0 + \text{valA}$	Pass valA through ALU
	rmmovq rA, D(rB)	
Execute	$\text{valE} \leftarrow \text{valB} + \text{valC}$	Compute effective address
	popq rA	
Execute	$\text{valE} \leftarrow \text{valB} + 8$	Increment stack pointer
	jXX Dest	
Execute		No operation
	call Dest	
Execute	$\text{valE} \leftarrow \text{valB} + -8$	Decrement stack pointer
	ret	
Execute	$\text{valE} \leftarrow \text{valB} + 8$	Increment stack pointer

Memory Logic



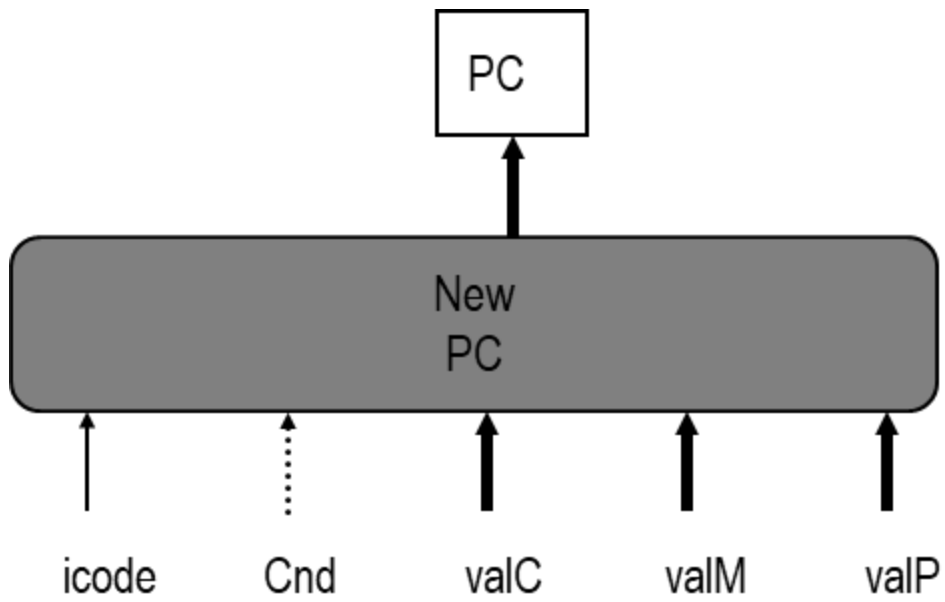
Memory:

- read or write memory words

Control logic:

- stat: the instruction status
- Mem. read: whether the word should be read
- Mem. write: whether the word should be written
- Mem. addr: Select address
- Mem. data: Select data

PC Update Logic



New PC:

- Select next value of PC

Classwork

Problem 4.13